Loan Approval Prediction Using Random Forests

Aryaf Elaqi , Reem Badri , Arwa Qumayri , Afnan Aqdi
Computer Science , College of Engineering and Computer Science
Jazan University

***Abstract*— Personal loans are a critical financial tool for individuals who need funding for education, medical treatment, business ventures, or other expenses, and wrong approval decisions can generate high financial risk for banks and lenders. Machine learning, which is the field that builds models able to learn patterns from data, can help automate and improve loan decision-making by discovering complex relationships in historical customer data rather than relying only on manual human judgment. In this work, a complete data-driven pipeline is developed that analyzes loan applicants' demographic, financial, and credit information in order to predict whether a loan should be approved or rejected. The dataset contains 45,000 records and 14 features, where a feature is any measurable characteristic of an applicant such as income, credit score, or education level. After appropriate data preprocessing, including the encoding of categorical variables into numerical form and the scaling of numerical variables to comparable ranges, a Random Forest classifier is trained. A Random Forest is an ensemble model composed of many decision trees that vote together on the final prediction, which reduces overfitting and increases robustness. In addition to simply reporting the overall accuracy, the model is evaluated using a detailed confusion matrix and a Receiver Operating Characteristic (ROC) curve in order to understand its behavior on both approved and rejected classes. The proposed model achieves an accuracy of about 92.56% on the test set, with strong precision and recall for both approved and rejected classes, and an Area Under the ROC Curve (AUC) of approximately 0.97, which indicates excellent discrimination capability between the two classes. The experimental results demonstrate that tree-based ensemble methods can effectively support loan approval decisions and reduce credit risk. Furthermore, several directions are discussed for handling class imbalance, for enhancing feature engineering, and for better capturing rare but important default cases in future versions of the project.**

Keywords*— *Loan approval, data preprocessing, Random Forest, classification, credit risk, machine learning.

I. Introduction

Loan approval is a key process in the banking and financial sector where institutions must decide, usually under time pressure and with incomplete information, whether to grant or reject a client's request for credit. A wrong decision can either reject a financially sound customer, which leads to lost business opportunities and reduced profit, or approve a high-risk customer who may default, which causes direct financial loss and increased regulatory risk. From the bank's perspective, every accepted customer who later fails to repay represents a realization of credit risk, while every rejected but trustworthy customer represents an opportunity cost. Traditionally, loan officers base decisions on a combination of credit scores, income, employment history, and personal experience. These decisions may also be influenced by implicit judgment and qualitative impressions, which are difficult to standardize or audit. However, as the number of applications grows and regulations push for more objective justifications, relying only on manual assessment becomes slow, expensive, and prone to human bias and inconsistency. This motivates the exploration of automated and data-driven approaches that can support or complement human decision-makers.

A machine learning model is a mathematical function trained on historical examples that maps input features, such as income and loan amount, to an output label, such as approved or rejected. Once trained, the model can automatically predict the label for new, unseen applications based on patterns it has learned from the past data. This approach reduces subjectivity, enables consistent decision-making, and can capture highly non-linear relationships between features and the probability of default that are difficult for humans to detect. For example, the interaction between income, loan_percent_income, credit_score, and previous_loan_defaults_on_file may jointly influence risk in ways that are not obvious from any single feature in isolation. In addition, machine learning makes it possible to monitor performance quantitatively by using objective metrics such as accuracy, precision, recall, and F1-score, and to systematically evaluate the impact of model changes over time. In this project, the chosen algorithm is Random Forest, which is an ensemble of decision trees where each tree is built on a random subset of the data and features and where the final prediction is obtained by majority vote. This type of model handles mixed numerical and categorical data, is relatively resistant to outliers—data points that are unusually far from the rest of the observations—and usually performs well with limited hyperparameter tuning, making it a practical choice for a first modeling attempt in applied projects like this one.

A publicly available loan dataset with 45,000 instances is analyzed using a standard data science workflow. The data is first explored using descriptive statistics and visualizations in order to understand the distributions of key variables such as credit score, monthly income, loan intent, and loan status. Several plots, including histograms, bar charts, a pie chart, and a scatter plot, are produced to reveal patterns and potential anomalies. These figures also help verify that the data is reasonable and matches expectations for a real-world lending scenario, for example by checking that most applicants have positive income, that credit scores fall into plausible ranges, and that loan amounts are not negative or extremely unrealistic. After that, data preprocessing is carried out: categorical features are converted into numerical form by one-hot encoding, numerical features are standardized so that they share a similar scale, and a logarithmic transformation is applied to the income variable to reduce skewness. The cleaned data is then split into training and testing subsets, and a Random Forest classifier with 300 trees is trained. The model is evaluated using a confusion matrix, which summarizes correct and incorrect predictions, and a Receiver Operating Characteristic (ROC) curve, which shows the trade-off between the true positive rate and the false positive rate for different thresholds. The main contribution of this project is the design of a complete and well-documented pipeline,

including clear explanations of all technical terms and detailed interpretation of all generated figures, that demonstrates how machine learning can be used to support loan approval decisions. In doing so, the project provides a concrete example of how theory from the literature on Random Forests [1] and model evaluation [3] can be applied to a realistic credit scoring problem using standard tools [2], [4].

II. METHODOLOGU

A. Dataset Description

We use a tabular dataset containing 45,000 rows (records) and 14 columns (features). Each row represents one loan application, and the target variable loan_status indicates whether the loan was approved (1) or rejected (0). A dataset is a structured collection of data used for analysis or model training, and a feature is a measurable property or characteristic of each instance (e.g., age, income, credit score).

Table 1 summarizes the features used in the project. The person_age feature records the age of the applicant in years. Age is a useful indicator because, in practice, creditworthiness is often related to stability of employment and life stage. The person_gender feature is a categorical variable indicating the gender of the applicant, stored as text, and later encoded numerically. The person_education feature represents the highest level of education obtained by the applicant, such as High School, Bachelor, or Master. Higher education levels may correlate with higher earning potential and more stable career paths. The person_income feature stores the annual income of the applicant before the logarithmic transformation is applied, and thus is an important measure of repayment capacity. The person_emp_exp feature indicates the number of years of employment experience, which again signals stability and reliability in income.

The person_home_ownership feature describes the type of home ownership, such as RENT, OWN, or MORTGAGE. This variable can provide indirect information about wealth and long-term financial obligations. The loan_amnt feature expresses the requested loan amount, which directly affects the level of risk because larger loans pose greater potential loss in the event of default. The loan_intent feature captures the purpose of the loan, for example EDUCATION, MEDICAL, PERSONAL, or VENTURE. Different intents can be associated with different risk profiles: for instance, debt consolidation loans might be riskier than education loans. The loan_int_rate feature indicates the interest rate, which in real applications may be adjusted by the lender based on perceived risk, although in this dataset it is treated as a given numeric feature. The loan_percent_income feature, which is the ratio of loan_amnt to person_income, is a derived indicator of how heavy the loan burden is relative to the applicant's earning capacity.

The cb_person_cred_hist_length feature expresses the length of the applicant's credit history in years. A longer credit history usually provides more information about repayment behavior and can be a sign of financial maturity. The credit_score feature gives the credit bureau score for each applicant, usually on a scale such as 300–850, and is one of the most important variables in traditional credit scoring systems. The previous_loan_defaults_on_file feature is categorical and indicates whether the applicant has a history of defaulting on prior loans. This is a crucial risk factor because past default behavior is a strong predictor of future default. Finally, loan_status is the target label that the model seeks to predict. The dataset contains no missing values (empty or null entries) and no duplicate rows (exactly repeated records), which simplifies preprocessing because we do not need to impute missing data or remove repeated records. This clean structure allows us to focus our efforts on modeling rather than on heavy data cleaning.

Table 1 : Describe the dataset features

Feature name	Describtion
person_age	Age of the applicant in years (numeric feature)
person_gender	Gender of the applicant (e.g., male, female) – categorical text.
person_education	Education level (e.g., High School, Bachelor, Master).
person_income	Annual income of the applicant before log transform.
person_emp_exp	Employment experience in years (integer).
person_home_ownership	Type of home ownership (RENT, OWN, MORTGAGE, etc.).
loan_amnt	Requested loan amount in monetary units.
loan_intent	Purpose of the loan (EDUCATION, MEDICAL, PERSONAL, VENTURE, etc.).
loan_int_rate	Interest rate for the loan (percentage).
loan_percent_income	Ratio of loan amount to person income (loan_amnt / person_income).
cb_person_cred_hist_length	Length of the applicant's credit history in years (numeric).
credit_score	Credit score from a credit bureau (integer, usually 300–850).
previous_loan_defaults_on_file	Whether the applicant has previous loan defaults on file (Yes/No).
loan_status	Target label: 1 = loan approved, 0 = loan rejected.

The dataset contains no missing values (empty or null entries) and no duplicate rows (exactly repeated records), which simplifies preprocessing. This means that every record contributes unique information to the model and that all variables are fully observed. As a result, we do not need to apply imputation techniques, which are methods for filling in missing values using statistics such as the mean or median, as described in general data mining literature [4].

B. Dara Preprocessing

Data preprocessing is the stage where raw data is transformed into a clean and suitable format for training a machine learning model. The process starts with descriptive statistics computed using the describe() function in the code. Descriptive statistics provide summary measures such as the mean, which is the arithmetic average of values, the standard deviation, which quantifies how spread out the data is around the mean, and the minimum and maximum values, which show the range of the variable. For example, the mean age in the dataset is about 27.76 years, and the mean credit score is about 632.6, which already suggests that the typical applicant is a relatively young adult with a medium credit score. These statistics help identify extremely large or small values and provide a first impression of potential outliers. An outlier is defined as a data point whose value is much larger or much smaller than most of the other observations, and it can exert a strong influence on models and summary statistics if not handled properly [5].

The next important preprocessing step is the log transformation of income. In the code, the function np.log1p(df["person_income"]) is applied, which computes the natural logarithm of one plus the income value. A logarithmic transformation is useful when the original distribution of income is highly skewed, meaning that most people earn moderate amounts while a small number earn very high incomes. Without transformation, such high values can dominate the scale and make it difficult for models to learn the patterns of the majority. After the transformation, the distribution becomes more symmetric and closer to a normal shape, and absolute differences between low and high incomes are compressed. This also helps to reduce the influence of income outliers, where an outlier is defined as a data point whose value is much larger or much smaller than the rest and that can potentially distort statistical analysis and model fitting. In the EDA stage, the difference between the raw and log-transformed income distributions can be clearly seen by comparing their histograms, which motivates the use of the transformation as part of preprocessing.

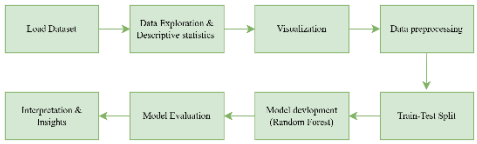
Categorical variables, which take textual values such as "Male", "Female", "RENT", or "OWN", cannot be directly processed by most machine learning algorithms that expect numerical input. To address this, one-hot encoding is applied using OneHotEncoder(handle_unknown="ignore"). One-hot encoding creates a new binary feature for each category, where the value is 1 if the category is present and 0 otherwise. For instance, person_gender is transformed into two new columns, one representing "male" and one representing "female". The argument handle_unknown="ignore" ensures that if a previously unseen category appears in the test set, it does not cause an error; instead, it is simply ignored. This conversion creates a purely numerical feature matrix that is compatible with the Random Forest classifier. Although one-hot encoding increases the dimensionality of the data set by adding additional columns, Random Forest models are known to cope well with such expanded feature spaces because each tree considers only a random subset of features at each split [1].

In addition to encoding, numerical features are standardized using StandardScaler(). Standardization is a procedure that rescales each numerical feature so that it has mean 0 and standard deviation 1. This is especially beneficial when features have different units or ranges, for example when comparing age measured in years to income measured in large currency units. Although Random Forests are less sensitive to scaling than some other algorithms, the project maintains consistency by including this step. Standardization also improves the interpretability of some plots and statistical summaries, because it puts all features on a common scale and can sometimes reveal relationships that are obscured when raw scales differ dramatically. A ColumnTransformer is used to apply the appropriate transformations to each group of columns in a single unified step, which ensures that the preprocessing is applied identically to training and testing data.

Finally, the dataset is split into training and testing subsets using train_test_split with a 70/30 ratio and the stratify option. The training set is used to fit, or learn, the model parameters, while the test set is reserved exclusively for evaluating model performance on unseen data. Stratification maintains the same proportion of approved and rejected loans in both sets as in the original data, which is crucial because the dataset is imbalanced; most applications are rejected (class 0) and a smaller portion are approved (class 1). Class imbalance is a common challenge in real-world credit scoring, and if it is not considered, a classifier may learn to predict the majority class most of the time and still achieve high accuracy. By using stratification, the model is at least exposed to a similar class distribution during training and evaluation, which provides a fairer view of its performance on the minority class of approved loans [4].

C. Proposed Method

Figure 1. Overall Machine Learning Workflow Diagram.



This figure illustrates the full end-to-end workflow followed in the project, beginning with loading and investigating the dataset, continuing with data preprocessing, then splitting into training and testing sets, and finally training and evaluating the Random Forest classifier. The workflow emphasizes that model building is not a single step but a sequence of interconnected tasks, where errors or poor choices early in the pipeline, such as inadequate preprocessing, can negatively affect model performance later on. By following a clear structure, the project aligns with best practices in applied predictive modeling [4].

A Random Forest classifier, implemented as RandomForestClassifier(n_estimators=300, random_state=42), constructs a large collection of decision trees during training. A decision tree is a model that predicts the target by repeatedly splitting the data according to feature values using if-then rules. At each internal node in a tree, a decision rule such as "credit_score < threshold" or "loan_percent_income > threshold" is chosen to best separate the classes according to an impurity measure, such as Gini impurity or entropy. Each tree in the forest is trained on a bootstrap sample, which is a random sample with replacement of the training data, and uses a random subset of features at each split. This randomness decorrelates the trees, and the final prediction is obtained by majority vote among all trees. This ensemble approach reduces overfitting, which is the situation where a model fits the noise in the training data instead of the underlying pattern, and yields a more stable and accurate model. The parameter n_estimators=300 specifies that the forest contains 300

trees, and random_state=42 is a fixed seed for the random number generator to make the results reproducible.

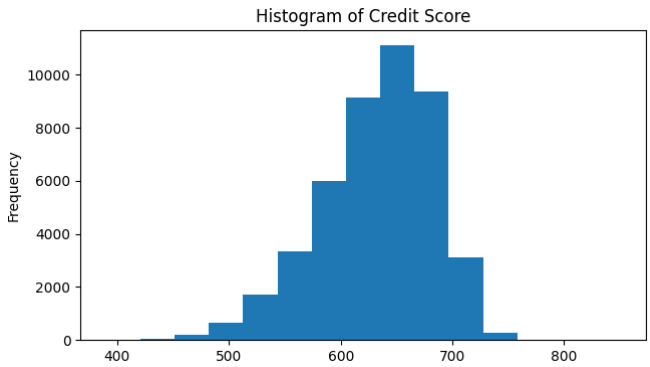
In the implementation, a pipeline object is used to ensure a proper sequence of operations. A pipeline is an ordered chain of transformations and a final estimator, so that the same steps are automatically applied whenever data is passed through. In this project, the core preprocessing using the column transformer is applied to produce prepared feature matrices X_train_prepared and X_test_prepared, and then the Random Forest model is trained and evaluated on these matrices. Training, also called fitting, is carried out by calling rf_model.fit(X_train_prepared, y_train), where y_train contains the true loan_status labels for the training examples. After training, predictions for the test set are obtained with rf_model.predict(X_test_prepared), which returns class labels 0 or 1, and rf_model.predict_proba(X_test_prepared), which returns estimated probabilities of loan approval. These probabilities are essential for constructing the ROC curve because the ROC curve shows how the model behaves as the classification threshold varies from 0 to 1. The use of a probabilistic output instead of only hard class labels also opens the door to calibrating decision thresholds depending on the bank's risk appetite, for instance by requiring a higher probability for approval if the institution wants to be more conservative.

III. Results Discussion and Limitation

A. *Algorithms results*

1. Credit Score Histogram

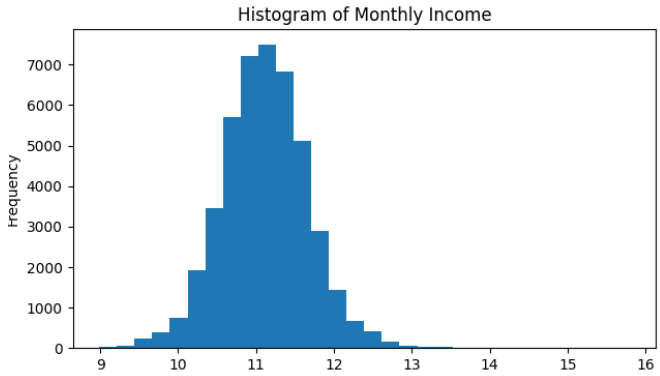
Figure 2. Distribution of Credit Scores.



This histogram shows how applicants' credit scores are distributed. Most applicants fall between 600 and 700, with a concentration around 650. A histogram groups numeric values into bins and displays their frequencies, helping identify clusters and outliers. The relatively smooth shape of the distribution suggests that the credit scores in the dataset cover a wide but realistic range of applicants, from somewhat risky to relatively safe borrowers. Very low scores are rare, which may mean that extremely risky customers are filtered out before applying, while very high scores are also relatively uncommon, indicating that the dataset mainly concentrates on average borrowers rather than exceptionally low-risk individuals. Because credit score is a core indicator of creditworthiness in traditional banking practice, this figure confirms that the dataset includes meaningful variation in risk that the Random Forest model can potentially exploit.

2. Log-Transformed Income Histogram

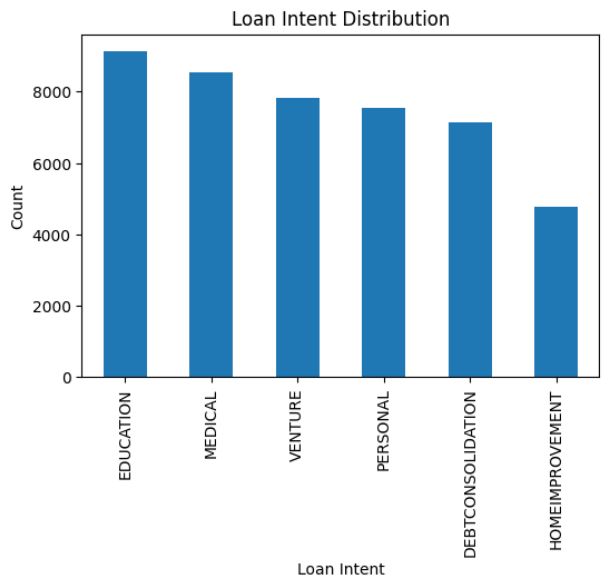
Figure 3. Distribution of Log-Transformed Income.



The log transformation reduces skewness in the income feature. Most log-income values fall between 10 and 12, producing a nearly symmetric bell-shaped distribution. This reduces the effect of income outliers—values that are unusually distant from the majority. Before transformation, a small group of very high-income applicants could have distorted the scale and dominated the variance of the feature. After transformation, the distribution becomes more balanced, and the difference between moderate and high incomes is compressed, which allows the model to better focus on patterns relevant for the majority of applicants. The histogram also confirms visually that there are no strange or unrealistic values, such as negative incomes. By using the transformed income rather than the raw income, the Random Forest model receives a feature that carries the same ordering of applicants from low to high income but in a numerically more stable form.

3. Loan Intent Distribution

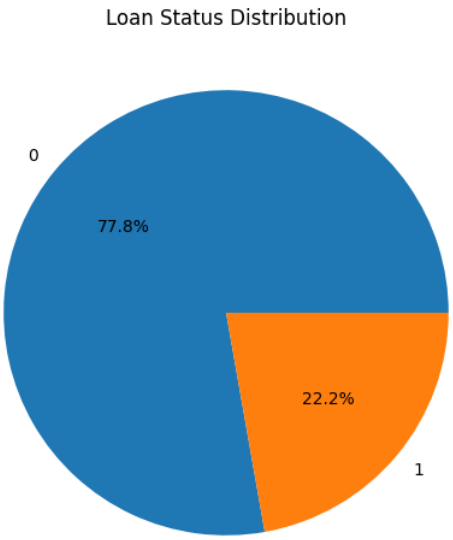
Figure 4. Distribution of Loan Intent Categories.



This bar chart shows how many applicants requested loans for each purpose. Education is the most common intent, followed by medical and venture categories, while home improvement is the least common. The heights of the bars represent the counts of applications in each category, and they immediately reveal which loan products are most in demand in the dataset. The dominance of education and

medical intents suggests that many applicants are using loans for long-term human capital investment or essential health expenses rather than purely discretionary spending. This information is not only descriptive but also relevant for modeling, because different intents may carry different risk levels; for example, applicants taking loans for business ventures may face different default risks than those consolidating debt. When the Random Forest learns patterns, it can implicitly take advantage of these categorical differences because loan_intent is one of the input features that undergo one-hot encoding.Loan Status Distribution

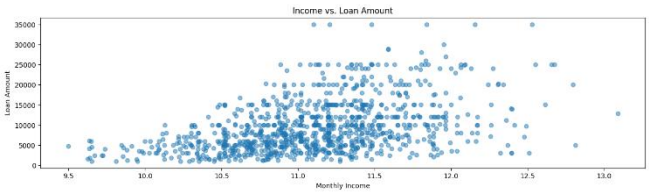
Figure 5. Loan Status Distribution (Approved vs Rejected).



This pie chart reveals a strong class imbalance, with around 77.8% rejected and only 22.2% approved. Class imbalance occurs when one category greatly outnumbers the other, leading models to favor the majority class. The chart makes clear that simply predicting all loans as rejected would already achieve relatively high accuracy, but such a model would be useless from a business perspective because it would never identify good customers. The figure therefore provides essential context when interpreting the performance metrics of the Random Forest model: high accuracy alone is not sufficient; performance on the minority approved class must also be considered. In practical deployments, handling this imbalance might involve resampling techniques or adjusting class weights, as discussed in the limitations section and in the predictive modeling literature [4].

4. Income vs Loan Amount Scatter Plot

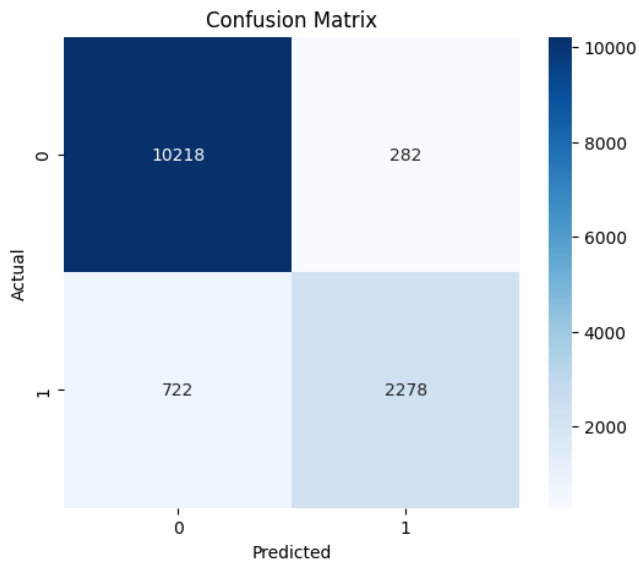
Figure 6. Scatter Plot of Log Income vs Loan Amount.



This scatter plot shows the relationship between applicants' income levels (log-transformed) and the requested loan amounts. Most applicants cluster around moderate income and loan values, forming a dense cloud in the central region of the plot. Some isolated high-value points represent outliers—unusually large loan requests. The general pattern suggests that, as expected, higher-income applicants tend to request larger loans, although there is significant variation. This relationship indicates that loan_amnt and person_income (or loan_percent_income) together can provide important explanatory power when predicting loan_status. For example, a high loan_percent_income combined with a low credit_score might indicate a high-risk situation, whereas a moderate loan_percent_income combined with a high credit_score might be considered safer. The scatter plot helps visually validate that the data behaves in line with economic intuition before relying on the automatic learning capabilities of the Random Forest model.

The performance of the Random Forest model is evaluated using standard classification metrics computed on the test set. The central tool for understanding these results is the confusion matrix, which is a two-by-two table that counts correct and incorrect predictions for each class. In the confusion matrix of this project, there are 10,218 true negatives, which are rejected loans correctly predicted as rejected, and 2,278 true positives, which are approved loans correctly predicted as approved. There are 282 false positives, where the model incorrectly predicts approval for loans that were actually rejected, and 722 false negatives, where the model incorrectly predicts rejection for loans that were actually approved. These numbers reflect the fact that the model is particularly strong at identifying rejected loans, which is consistent with the underlying class imbalance shown in the loan status distribution.

Figure 7. Confusion Matrix of the Random Forest Classifier.



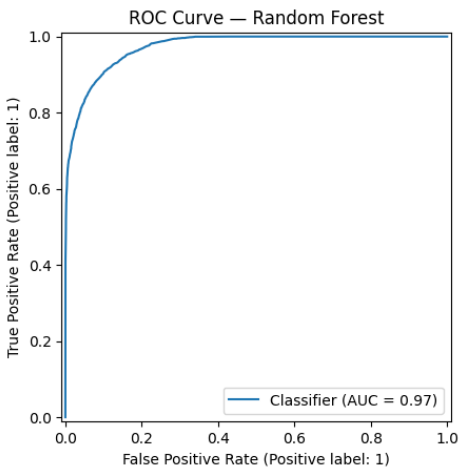
The confusion matrix summarizes correct predictions (true positives and true negatives) and misclassifications (false positives and false negatives). The heatmap shading highlights the high number of correct predictions on the diagonal. In the visual representation, darker cells appear along the diagonal, indicating that most of the predictions are correct, especially for the rejected class. Lighter cells in the off-diagonal positions represent errors. The asymmetry between the false

positives and false negatives indicates that the model tends to be slightly more conservative: there are more false negatives (missed approvals) than false positives (incorrect approvals). This behavior may actually be acceptable or even desirable in some banking contexts where the cost of approving a bad loan (false positive) is higher than the cost of rejecting a good loan (false negative), but such trade-offs must be aligned with the institution's risk policy.

From these counts, several scalar metrics are derived. Accuracy is the ratio of correct predictions to all predictions; it is computed as (TP + TN) divided by (TP + TN + FP + FN) and in this project equals approximately 0.9256, meaning that about 92.56% of all decisions on the test set are correct. Precision for the approved class is the fraction of predicted approved loans that are truly approved, given by TP divided by (TP + FP), and has a value of about 0.8898. Recall for the approved class, also called sensitivity or true positive rate, is the fraction of actual approved loans that are correctly identified, given by TP divided by (TP + FN), and equals about 0.7593. The F1-score, which is the harmonic mean of precision and recall and balances the trade-off between them, is about 0.8194. Together, these metrics show that the model is accurate overall, tends to have relatively few false approvals, and successfully identifies a large majority of approved applications, although some are still missed. They also indicate that the model leans slightly toward being cautious in approving loans, which again may be more acceptable than the opposite in a risk-sensitive environment.

To further assess discriminative power, the ROC curve is examined. The ROC, or Receiver Operating Characteristic, curve plots the true positive rate on the vertical axis against the false positive rate on the horizontal axis for all possible classification thresholds. The false positive rate is defined as FP divided by (FP + TN) and measures the fraction of rejected loans incorrectly predicted as approved. A model with no skill would produce a diagonal line from the bottom left to the top right of the ROC space, while a perfect model would pass through the top-left corner, indicating a true positive rate of 1 and a false positive rate of 0. The area under the ROC curve (AUC) condenses this information into a single number between 0 and 1. In this project, the AUC is approximately 0.9729, which indicates that, given a random approved applicant and a random rejected applicant, the model will assign a higher approval probability to the approved one about 97% of the time, showing excellent discriminatory ability [3].

Figure 8. ROC Curve with AUC = 0.9729.



The ROC curve plots the true positiverate against the false positive rate across different thresholds. The model's curve approaches the upper-left corner, indicating excellent discrimination. The AUC value of 0.97 confirms high predictive ability. Instead of being tied to a single threshold, the ROC curve shows how the model would behave if we became stricter or more lenient in classifying an applicant as approved. For example, moving the decision threshold upward would decrease the false positive rate but also lower the true positive rate, making the model more conservative. The existence of a high AUC value means that there exist thresholds at which both sensitivity and specificity are high, offering flexibility to adapt the model to various risk policies in practice.

B. Limitation

Despite the strong numerical and graphical results, there are several limitations and challenges in the project. The most significant limitation is the pronounced class imbalance in the dataset, where approximately three-quarters of the applications are labeled as rejected. This imbalance may bias the Random Forest model toward predicting rejection more often, which explains why recall for the approved class is lower than recall for the rejected class, even though overall accuracy is high. Techniques such as resampling the data, for example by oversampling the minority class or undersampling the majority class, or by assigning different class weights to penalize misclassifications of the minority more heavily, could mitigate this issue but were not explored in the current implementation. In future work, combining these strategies with more advanced ensemble algorithms could further improve the detection rate of approved loans without significantly increasing false approvals.

A second limitation is that only a single type of classifier, namely Random Forest, is considered. While Random Forests are powerful and versatile, other algorithms like gradient boosting machines, XGBoost, or logistic regression might produce different and potentially better trade-offs between interpretability, computational cost, and predictive performance. Logistic regression, for example, is easier to interpret but may not capture complex non-linear interactions as effectively. Gradient boosting methods can sometimes provide higher accuracy but at the cost of more careful tuning and a higher risk of overfitting if not properly regularized. Evaluating and comparing multiple models could provide a more comprehensive understanding of the problem and help identify the best algorithm for specific business goals.

Third, the project relies mostly on straightforward transformations of the original features and does not include extensive feature engineering. Feature engineering is the process of constructing new features from existing ones, for example combining income, credit score, and employment length into composite risk indicators. More advanced feature engineering could help the model better capture subtle aspects of creditworthiness. For instance, ratios between loan amount and credit history length, or interactions between loan intent and credit score, might reveal important patterns that are only partially captured in the current setup.

In many real-world data mining applications, feature engineering is one of the most impactful steps for improving model performance [4].

Finally, the dataset is treated as static, and temporal effects are ignored. In reality, economic conditions, interest rates, and customer behavior change over time, so a model trained on past data may degrade as conditions shift. Incorporating time information and periodically retraining or updating the model could lead to more robust long-term performance. Furthermore, fairness and ethical considerations, such as ensuring that the model does not inadvertently discriminate against protected groups, are not addressed in this project but would be essential in a real deployment. These aspects illustrate that high performance metrics on historical data are a necessary but not sufficient condition for responsible use of machine learning in credit decision-making.

IV. Conclusion

In this work, a full machine learning pipeline for loan approval prediction has been implemented and analyzed. Starting from a structured dataset of 45,000 loan applications with 14 features, the data was carefully explored using descriptive statistics and multiple visualizations, including histograms, bar charts, a pie chart, and a scatter plot, which together provided insight into the distribution of credit scores, incomes, loan intents, loan statuses, and the relationship between income and requested loan amount. Data preprocessing steps such as log transformation of income, one-hot encoding of categorical variables, and standardization of numerical features transformed the raw dataset into a form suitable for modeling.

A Random Forest classifier with 300 trees was trained on the preprocessed data and evaluated on a separate test set. The model achieved high accuracy (92.56%), strong precision and recall for the approved class, and an AUC close to 0.97, demonstrating excellent discrimination between approved and rejected loans. The confusion matrix and ROC curve provided detailed perspectives on different types of errors and trade-offs, while the explanations of all figures helped interpret the patterns learned by the model. The project thus shows how widely used methods from the machine learning literature [1]–[4] can be applied in a coherent way to a realistic loan approval scenario.

Although the findings show that Random Forests are effective tools for supporting loan approval decisions, the limitations related to class imbalance, the use of a single algorithm, limited feature engineering, and the absence of temporal modeling suggest several promising directions for future work. Future extensions could experiment with rebalancing strategies, compare different classifiers, design more informative features, and incorporate time-dependent behavior. In addition, further work could investigate model calibration, threshold optimization for different risk profiles, and explainability techniques such as feature importance or partial dependence plots to provide more transparency about how decisions are made. Overall, the study confirms that data-driven methods can significantly enhance the loan approval process and provide a solid foundation for more advanced credit-risk analytics, while also highlighting the need for continuous refinement and careful consideration of practical, ethical, and regulatory constraints.

References

[1] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] T. Fawcett, “An Introduction to ROC Analysis,” Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.

[4] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques, 3rd ed., Morgan Kaufmann, 2012.

[5] C. C. Aggarwal, Outlier Analysis, 2nd ed., Springer, 2015.